

Fine-Tuning LLMs on Your Own Data

1. What is Fine-Tuning?

Fine-tuning is taking an existing LLM and training it on YOUR specific dataset so it becomes an expert in your domain.

What happens technically:

- You start with an already-good model (e.g., Llama-3, Mistral, GPT)
- You train it on your company's data
- It **updates some internal weights** to minimize errors on your dataset
- Now it understands your domain deeply

After fine-tuning, your model can:

- Respond with the **right tone** (polite, helpful, non-judgmental)
- Answer in the **right format** (verbose, point-wise, structured)
- Follow **guardrails** (block NSFW content, dangerous queries)
- Use the **right tools** (calculator, search, internal APIs)

The Fine-Tuning Process

Step 1: Define the Objective

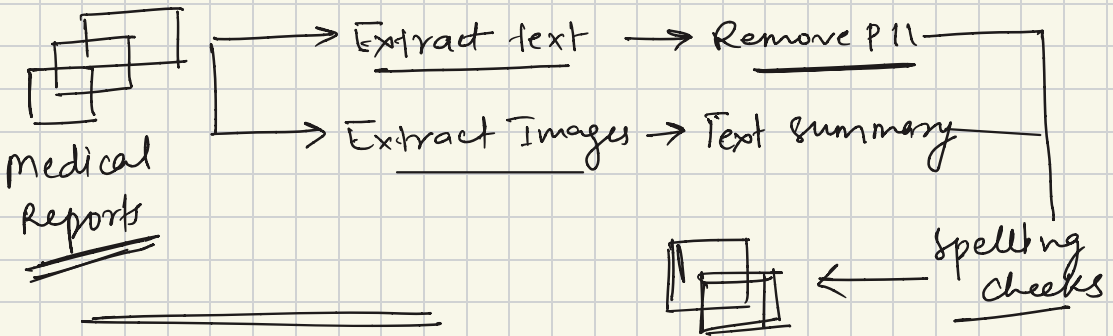
> Example: "Create diagnosis summaries from medical reports"

Step 2: Decide the Task

> Example: "Given medical data, find anomalies, describe their potential impact, causes, and mitigation plan"

Step 3: Prepare Your Data

Data must be **cleaned and transformed** before training:



Step 4: Train (Fine-tune)

Step 5: Evaluate

> Score the trained model on a validation set – does it actually do better?

Base LLM → experienced ^{chatbot}
general assistant

it knows

→ { english
→ how to answer ques
→ general HR concepts
→ How to write clear sentences. }

what it does not know

✓ { → your company's HR policy
→ your company's leave policy.
→ your WFH Rules. }

So you don't need to train a whole brain. You only need to teach it your company's cheat sheet

Types of Fine-Tuning

Type 1: Full Fine-Tuning (DON'T DO THIS)

What it does -> Updates ALL parameters of the model

Problem 1 -> Very compute heavy (slow & expensive)

Problem 2 -> Catastrophic forgetting – the model forgets what it already knew!

Used in practice? NO

Type 2: Parameter Freezing (Sometimes Used)

What it does -> Freezes most layers, updates only the last few

How it works:

- LLMs have many transformer layers stacked on each other
 - GPT-4 $\approx$ 120 layers
 - DeepSeek $\approx$ 60 layers
- You freeze most (e.g., 110 layers) and only update the last few (e.g., 10 layers)



Parameter efficient fine tuning (PEFT)

Type 3: LoRA – The Industry Standard (USE THIS)

LoRA = Low-Rank Adaptation

This is what everyone uses in practice for fine-tuning LLMs on custom data.

Understanding LoRA

A transformer has these components:

Embedding layer	~5%	→	params	} weights
Attention**	~25%	→	params	
FFNN (Feed-Forward)	~66%	→	params	
Linear layer	~1%	→	params	

Key finding: Updating just the **attention weights** is enough!

Even within attention, updating only **W_q (query weights)** is sufficient.

$$\text{softmax} \left(\frac{(\underbrace{W_q}_{\text{circled}} \cdot x) \cdot k}{\sqrt{d_k}} \right) \cdot V$$

= Attn

- W_q = only **~0.1%** of total parameters
- The remaining **99.9%** stays completely frozen

4 matrices

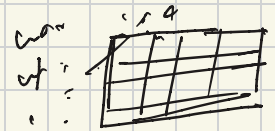
... $\underbrace{W_q}_{\text{circled}}$ → what am I looking for
 $\underbrace{W_k}_{\text{circled}}$ → what do I offer
 $\underbrace{W_v}_{\text{circled}}$ → what info I carry.
 $\underbrace{W_o}_{\text{circled}}$

How LoRA Works:

Core idea: Don't modify the original weights. Add a small "adapter" on top.

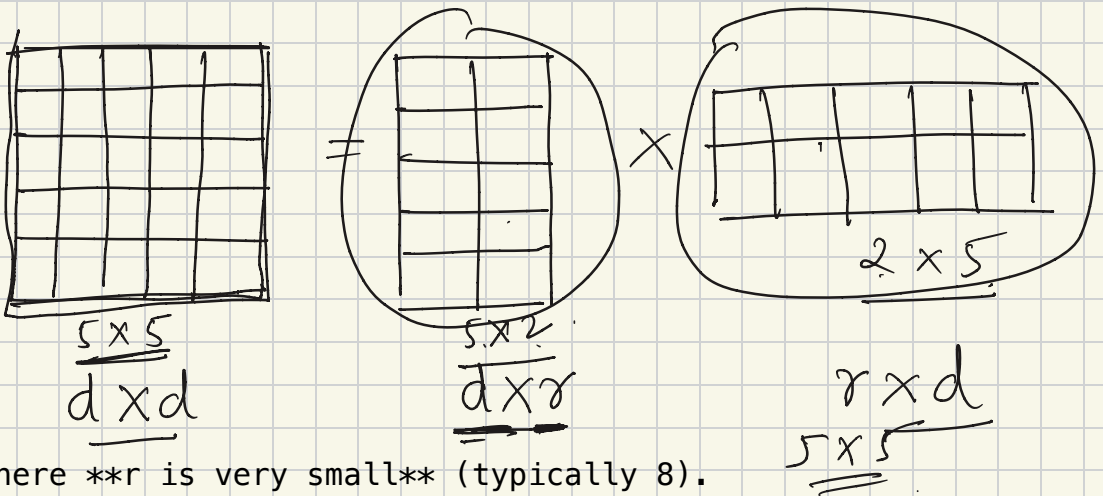
The original model stays untouched. The adapter is what learns your data.

The Math Trick (Low-Rank Decomposition):



A weight matrix might be $8192 \times 8192 = \text{**67 Million**}$ parameters.

Instead of training a full 67M adapter, LoRA decomposes it:

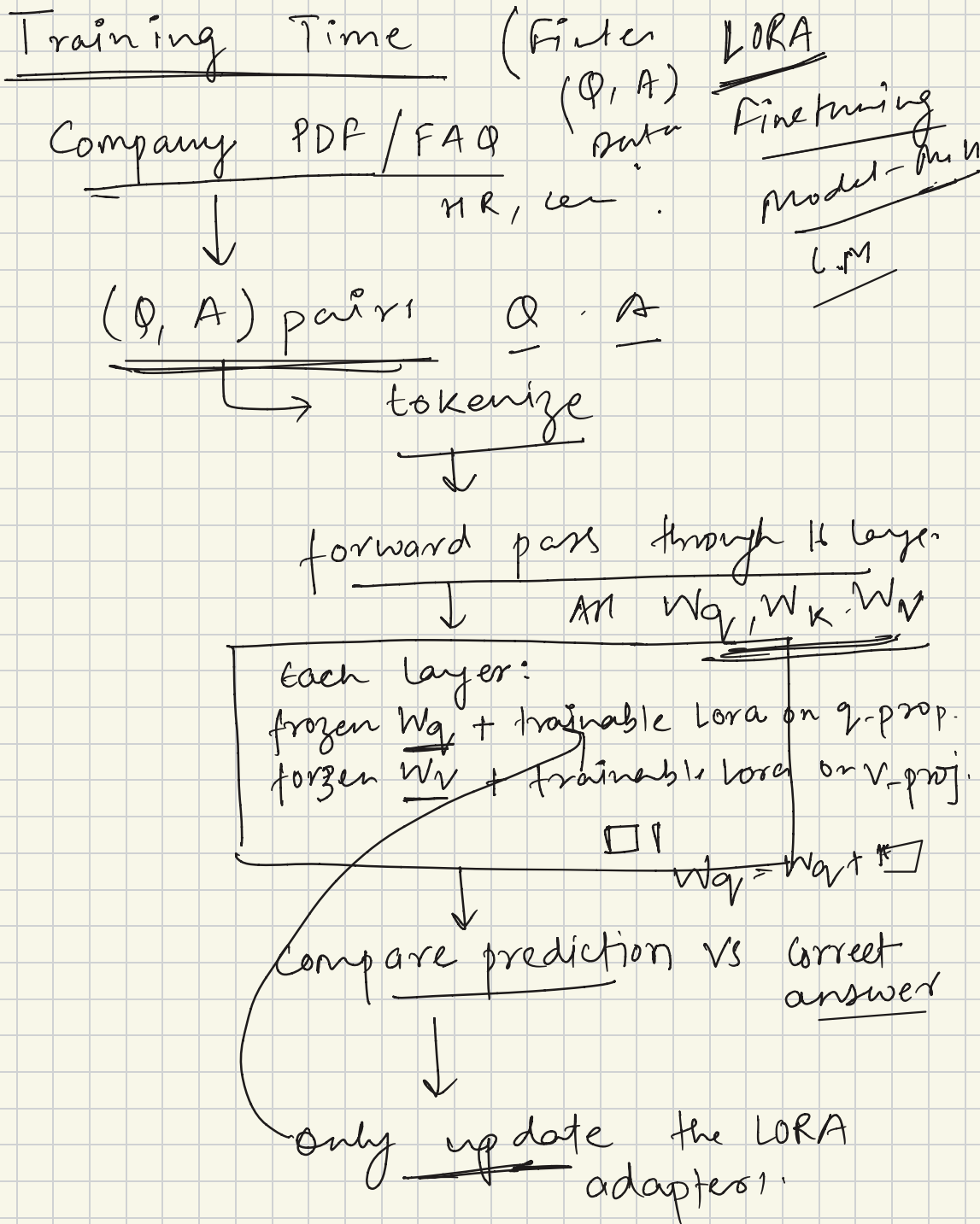


Where ****r is very small**** (typically 8).
Real example:

- Original: $8192 \times 8192 = 67\text{M}$ parameters
- LoRA: $8192 \times 8 \times 2 = 131\text{K}$ parameters
- That's only 0.2% of the original!

Training time:

- For large models: approximately 1 week (depends on dataset size)



QLoRA – Making It Even Cheaper

LoRA + Quantization

The Problem:

Even with LoRA, you need the **full model loaded in memory** (you're just not updating most of it).

Example

- 70B parameter model at 32-bit precision
- $70 \times 10^9 \times 4 \text{ bytes} = 280 \text{ GB VRAM}$
- That's multiple expensive A100 GPUs!

LoRA

The Solution – Quantization:

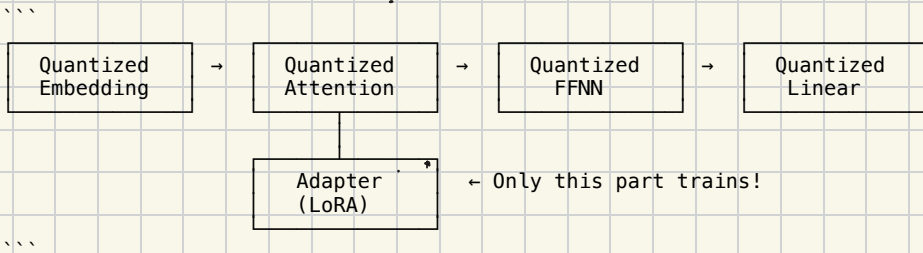
Imagine you're measuring people's heights.

- 32-bit = measuring with a laser: "1.82347291 meters" (super precise, but takes lots of storage)
- 8-bit = rounding: "about 1.8 meters" (less precise, but takes 4x less space)

For neural networks, each weight is a decimal number like `0.00234567891234`.`

- At 32-bit, we store all those decimal places exactly → 4 bytes per number
- At 8-bit, we round/compress it to fewer decimal places → 1 byte per number

QLoRA = Quantization + LoRA



Softwares used for Fine tuning

→ Unsloth → finetuning made faster

→ Llama-Factory

→ Lit GPT

→ OpenAI/Anthropic/Google APIs
(Managed fine-tuning)

Upload json L → they train
and host the model.

Disadvantages of Fine-Tuning

If your company's policies, pricing, or FAQ answers change:

- You have to **re-fine-tune** the model with updated data
- That means re-curating the dataset, re-running training, re-evaluating, and re-deploying
- Until you retrain, the model confidently gives **outdated answers**